

- What are the advantages of Polymorphism?

1. Tái sử dụng mã: Có thể sử dụng lại các phương thức cho nhiều lớp khác nhau giúp giảm thiểu sự trùng lặp
2. Tính linh hoạt và khả năng mở rộng: Có thể thêm các lớp mới mà không cần thay đổi trực tiếp vào mã nguồn hiện tại, giúp mở rộng hệ thống dễ dàng hơn
3. Dễ bảo trì: Khi cần chỉnh sửa, chỉ cần thay đổi một phần code giúp giảm thiểu rủi ro cho toàn bộ hệ thống
4. Đơn giản hóa mã nguồn: Có thể lập trình lớp cha, interface thay vì từng lớp con giúp mã nguồn ngắn gọn hơn
5. Giảm sự phụ thuộc: Bằng cách dựa vào interface và lớp trừu tượng thay vì các lớp cụ thể giúp làm giảm sự phụ thuộc giữa các thành phần trong hệ thống

- How is Inheritance useful to achieve Polymorphism in Java?

Kế thừa trở nên hữu ích để đạt được Đa hình bằng:

1. Lớp con có thể cung cấp một cách triển khai riêng cho phương thức đã được định nghĩa ở lớp cha
2. Tham chiếu của lớp cha có thể trỏ đến đối tượng của lớp con
3. Hỗ trợ tái sử dụng và mở rộng mã nguồn thông qua kế thừa các thuộc tính, phương thức, dễ dàng mở rộng hành vi của các lớp.

- What are the differences between Polymorphism and Inheritance in Java?

1. Về khái niệm:

- +) Kế thừa cho phép lớp con kế thừa lớp cha
- +) Đa hình cho phép đối tượng thể hiện nhiều hành vi khác nhau

2. Về mục tiêu:

- +) Kế thừa giúp tái sử dụng và tổ chức lớp
- +) Đa hình giúp viết mã linh hoạt và mở rộng

3. Về cách thực hiện:

+) Kế thừa: Lớp con sử dụng extends (implements với interface) để thừa hưởng thuộc tính và phương thức của lớp cha

+) Đa hình: Thể hiện thông qua ghi đè phương thức và nạp chồng phương thức.

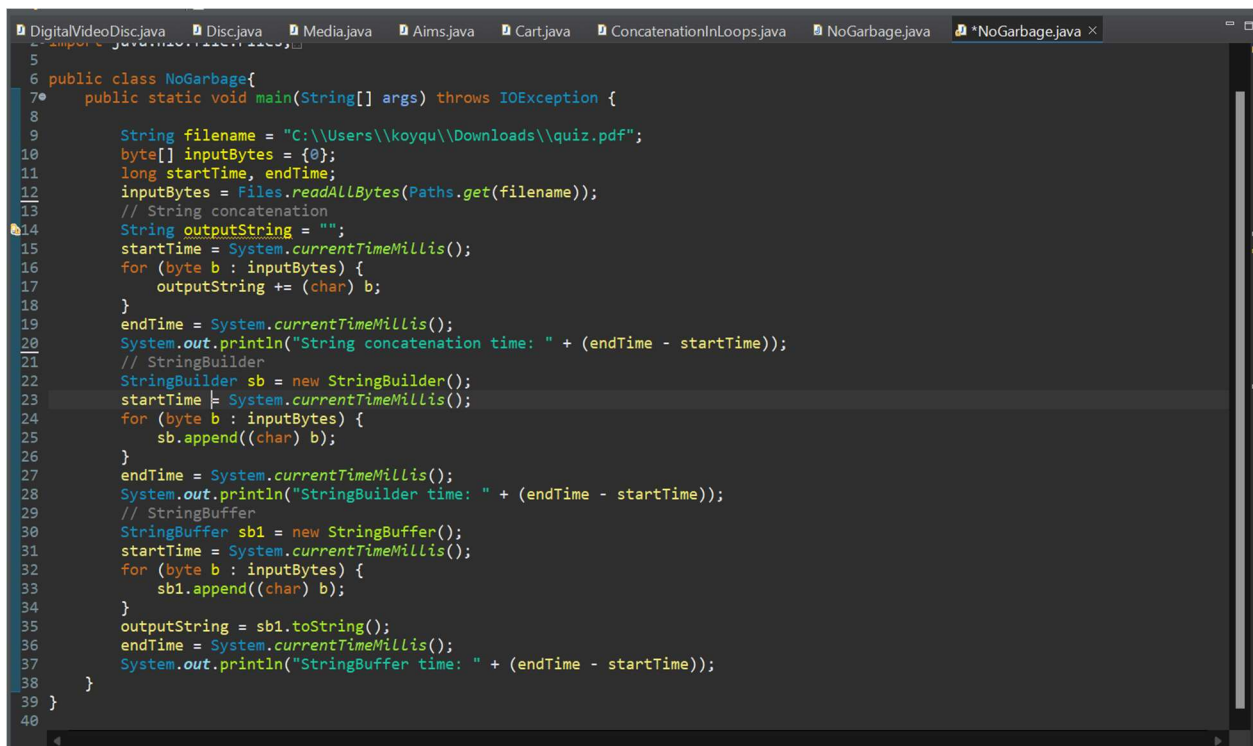
4. Về vai trò:

+) Kế thừa: Là nền tảng cho đa hình

+) Đa hình: Hoạt động trên cơ sở kế thừa

So sánh xử lý xâu:

Source code:



```
1 import java.io.*;
2 import java.nio.file.*;
3
4 public class NoGarbage {
5     public static void main(String[] args) throws IOException {
6         String filename = "C:\\Users\\koyqu\\Downloads\\quiz.pdf";
7         byte[] inputBytes = Files.readAllBytes(Paths.get(filename));
8         // String concatenation
9         String outputString = "";
10        long startTime, endTime;
11        startTime = System.currentTimeMillis();
12        for (byte b : inputBytes) {
13            outputString += (char) b;
14        }
15        endTime = System.currentTimeMillis();
16        System.out.println("String concatenation time: " + (endTime - startTime));
17        // StringBuilder
18        StringBuilder sb = new StringBuilder();
19        startTime = System.currentTimeMillis();
20        for (byte b : inputBytes) {
21            sb.append((char) b);
22        }
23        endTime = System.currentTimeMillis();
24        System.out.println("StringBuilder time: " + (endTime - startTime));
25        // StringBuffer
26        StringBuffer sb1 = new StringBuffer();
27        startTime = System.currentTimeMillis();
28        for (byte b : inputBytes) {
29            sb1.append((char) b);
30        }
31        outputString = sb1.toString();
32        endTime = System.currentTimeMillis();
33        System.out.println("StringBuffer time: " + (endTime - startTime));
34    }
35 }
```

Thời gian chạy của 3 cách làm:

String concat: 2520ms

String Builder: 1ms

String buffer: 45ms

```
String concatenation time: 2520  
StringBuilder time: 1  
StringBuffer time: 45
```

Hệ điều hành: Windows 11, 23H2 (64bit)

Dữ liệu test: Đọc 1 File PDF nặng 97KB

Phần mềm chạy: Eclipse IDE 2024-12 (4.34.0)

Java: JDK 1.8 (Java SE 8)

Cấu hình máy tính:

+) CPU: AMD Ryzen 5 6600H (6 cores / 12 threads)

+) GPU: RTX 3050 Laptop (4GB Vram)

+) Ram: 16GB DDR5 (2x8GB Dual Channel)

+) SSD NVMe 512GB

Đánh giá:

Trong cả 3 cách làm trên, chậm nhất là cách nối chuỗi thông thường dùng toán tử + với String do mỗi lần nối chuỗi JVM đều phải tạo ra một đối tượng mới và sao chép toàn bộ nội dung cũ dẫn tới tốc độ thực thi lớn. Hai cách String Builder và String Buffer rất nhanh. Tuy nhiên, String Builder vẫn nhanh hơn khá nhiều do đây là môi trường đơn luồng và không cần đồng bộ, mà String Buffer hiệu quả với môi trường đa luồng và có chi phí đồng bộ hóa khiến cho tốc độ trở nên chậm hơn so với String Builder.